
```
>>> def rectanglePerimeter(rect):  
...     return (rect[0] * 2) + (rect[1] * 2)  
...  
>>> myRectangle = [4, 10]  
>>> rectanglePerimeter(myRectangle)  
28
```

The equivalent lambda function would look like this:

```
lambda rect: (rect[0] * 2) + (rect[1] * 2)
```

To define a Python lambda function, use the `lambda` keyword, followed by a comma-delimited list of parameters (if any), a colon, and then an expression that acts as the return value. Because functions are first-class objects, you can assign a lambda function to a variable, effectively replicating what a `def` statement does:

```
>>> rectanglePerimeter = lambda rect: (rect[0] * 2) + (rect[1] * 2)  
>>> rectanglePerimeter([4, 10])  
28
```

We assigned this lambda function to a variable named `rectanglePerimeter`, essentially giving us a `rectanglePerimeter()` function. As you can see, functions created by `lambda` statements are the same as functions created by `def` statements.

NOTE

In real-world code, use `def` statements rather than assigning lambda functions to a constant variable. Lambda functions are specifically made for situations in which a function doesn't need a name.

The lambda function syntax is helpful for specifying small functions to serve as arguments to other function calls. For example, the `sorted()` function has a keyword argument named `key` that lets you specify a function. Instead of sorting items in a list based on the item's value, it sorts them based on the function's return value. In the following example, we pass `sorted()` a lambda function that returns the perimeter of the given rectangle. This makes the `sorted()` function sort based on the calculated perimeter of its `[width, height]` list rather than based directly on the `[width, height]` list: